Mixed Discrete-Continuous Optimization for Graph-Structured Models: Applications in Phylogenetics, Potts Models, and Hidden Markov Models

M. J. Wilson

January 1, 2025

Abstract

Inference over graph-structured models—phylogenetic trees, lattice-based Potts models, and hidden Markov models—is a coupled discrete-continuous optimization problem. The structural search space is combinatorial, while scoring each candidate structure requires continuously optimizing its parameters. We present a framework that strictly decouples differentiable continuous optimization from discrete structural search, so that one optimizer serves all three model classes and a structural move constructs a new objective rather than taking a step inside an existing one. By framing structural search as a Markov decision process, we enable reinforcement-learning agents to learn topological proposal policies against exact, approximate, or neural-surrogate objectives. We validate the engine against exact oracles—brute-force marginalization, path enumeration, and exhaustive topology enumeration—and report realized interval coverage rather than asserting it. We then outline the extensions required for curriculum learning, differentiable topology search, and canonical directed-acyclic-graph representations. Formulations, algorithms, and the properties each is validated against are given in the companion textbook; this paper reports results.

Contents

1 Introduction

The fundamental difficulty in high-dimensional probabilistic inference is the combinatorial size of the structural search space. In phylogenetics, the number of unrooted binary topologies over n taxa is $(2n-5)!!$ [7]. In statistical physics, the ground state of an N-dimensional Potts model requires traversing an exponentially large configuration space [4]. Decoding a hidden Markov model poses the same coupled problem on a chain [3].

Classical frameworks rely on fixed, hand-designed heuristics—nearest-neighbour interchange for trees, Swendsen–Wang cluster updates for lattices—applied greedily or under simulated annealing. This paper takes an alternative: the discrete search is a reinforcement-learning problem in which a neural policy proposes structural updates, while continuous parameters are fitted by reverse-mode automatic differentiation.

1.1 Scope

This paper reports what has been built and measured. The formulations it rests on—the substitution model, the Potts energy, the hidden Markov recursions, the discrete move sets, and the mathematical properties each algorithm is refereed by—are stated in the companion textbook rather than repeated here. **Empirical benchmarking of trained policies against state-of-the-art classical tools** (IQ-TREE 2, RAxML-NG) is slated for a later milestone and appears as marked placeholders in Section 3.

2 Methods

The methodology rests on one division: continuous parameters are optimized by gradients on a fixed structure, while discrete structural moves are proposed by an external agent and construct an entirely new differentiable objective. A move changes what the parameter vector means and how long it is, so it cannot be a step inside a fit over a fixed-length vector.

2.1 Continuous optimization

An objective is a differentiable scalar $\mathcal{L}(\boldsymbol{\theta})$ over an unconstrained vector $\boldsymbol{\theta} \in \mathbb{R}^d$, with an injective map ϕ to the constrained parameters the model is stated in. The same quasi-Newton optimizer [5] therefore minimizes negative log-likelihoods of phylogenetic alignments, free energies of Potts models, and sequence log-likelihoods of hidden Markov models without knowing which it has. Standard errors come from the observed information at the optimum, pushed through ϕ by the delta method.

2.2 Differentiable likelihoods and energies

Partial likelihoods are evaluated by a post-order recursion, with branch lengths $\boldsymbol{\tau}$ and the rate matrix $\mathbf{Q}$ kept inside the automatic differentiation graph, so gradients are exact rather than numerical [2, 6]. Rescaling against underflow is part of that graph.

2.3 Structural search as a Markov decision process

Discrete graph traversal is formulated as a Markov decision process [8]: the **state** is the current topology, its optimized continuous parameters, and observation summaries; an **action** is a structural perturbation from a classical neighbourhood; the **reward** is the improvement in the objective.

2.3.1 Planned architectural extensions

- **Neural surrogate modelling.** Rather than running exact inference on every proposal, a graph neural network or transformer approximates $\mathcal{L}$; the agent filters proposals with the surrogate and evaluates exactly only the top candidates.
- **Curriculum learning.** The policy is trained on small state spaces before transferring to progressively larger graphs, to avoid policy collapse.
- **Differentiable topology search.** Continuous relaxations embed the discrete structure in a manifold, enabling end-to-end topological updates by natural gradients [1].

3 Results

Every figure below is rendered from the code it reports on and carries a generated caption naming the seed, sizes, and model that produced it.

3.1 Simulation against its analytic truth

$$(A\_0.1, B\_0.25, (C\_0.15, D\_0.4)\alpha\_0.05)\rho$$

	0	1	2	3	4	5	6	7	8	9
A	T	G	A	T	G	G	G	T	A	G
B	A	G	T	T	T	T	G	T	A	G
C	T	G	A	T	T	G	G	T	A	G
D	G	G	A	T	C	T	G	C	T	A

Figure 1: Worked example: 4-taxon alignment simulated under the Jukes-Cantor model (seed 20260902, 200_000 sites total), showing the topology with branch lengths – rho the root, Greek letters its internal ancestors – and the first 10 of 200_000 simulated sites.

3.2 Exact inference across backends

3.3 Parameter recovery and interval coverage

Recovery is the acceptance test for a fit, and a Wald interval from the observed information is an *asymptotic* claim. The two figures below are the evidence for it rather than an assertion of it: the first shows each fitted parameter against the value that generated the data, the second shows the realized coverage against the amount of data each fit sees.

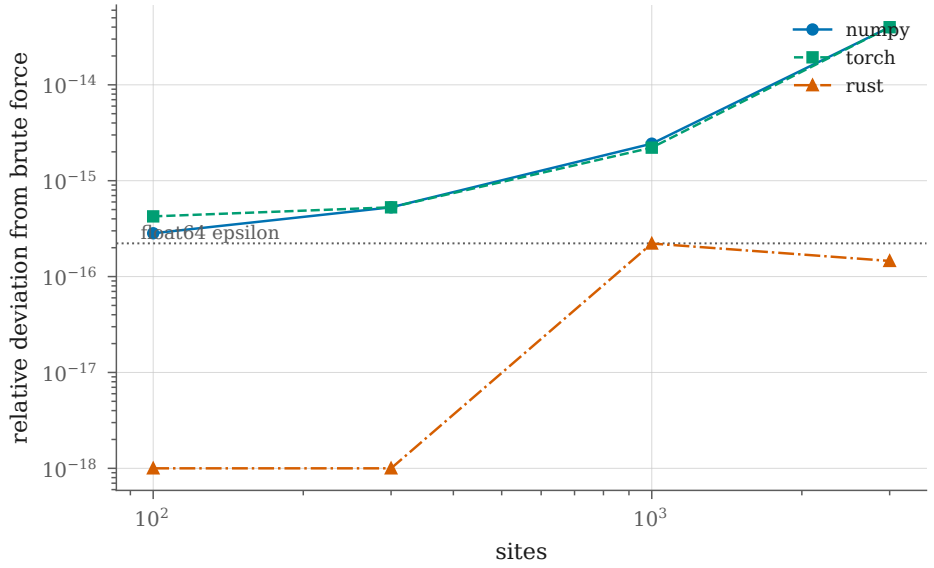

Figure 2: Every pruning backend against the oracle that owes it nothing. Brute-force marginalization sums over all ancestral assignments, so it shares no code with the recursion it checks, and its cost is what keeps the fixture small. Fixture: 4-state Jukes-Cantor on the 3-child-root tree, seed 20260902. The deviation is relative because the log-likelihood is a sum over sites: an absolute bound fixed at one site count does not transfer to another, which is why the x-axis spans a factor of 30. Worst deviation across every backend and size: 3.99e-14, against a float64 epsilon of 2.22e-16.

3.4 Discrete search against exhaustive enumeration

Below the size at which every topology can be enumerated, enumeration is the only independent statement available about whether a search found the best tree or merely a good one.

3.5 Memory footprint at the declared scale

The roadmap bounds the working set to $O(n \times L \times k)$ inside 16 GB of unified memory. Table 1 settles it, splitting the cost the data carries from the cost the evaluator adds: the simulator retains every node's states, while pruning retains a partial likelihood only for nodes whose parent has not yet consumed it and so tracks the tree's depth. Both are reported on a caterpillar, the deepest tree on its leaves, which makes the bound tight rather than loose and the reported total a worst case rather than a typical one.

3.6 Empirical benchmarking [PLACEHOLDERS]

The following are populated on completion of the benchmarking milestone.

4 Conclusions

Decoupling structural exploration from continuous fitting yields a model-agnostic optimization engine that serves phylogenetics, Potts models, and hidden Markov models without encoding any of them. Formulating graph search as a Markov decision process gives a foundation for learned topological proposals and neural surrogate modelling. Future work populates the benchmarking placeholders and demonstrates competitiveness at scale.

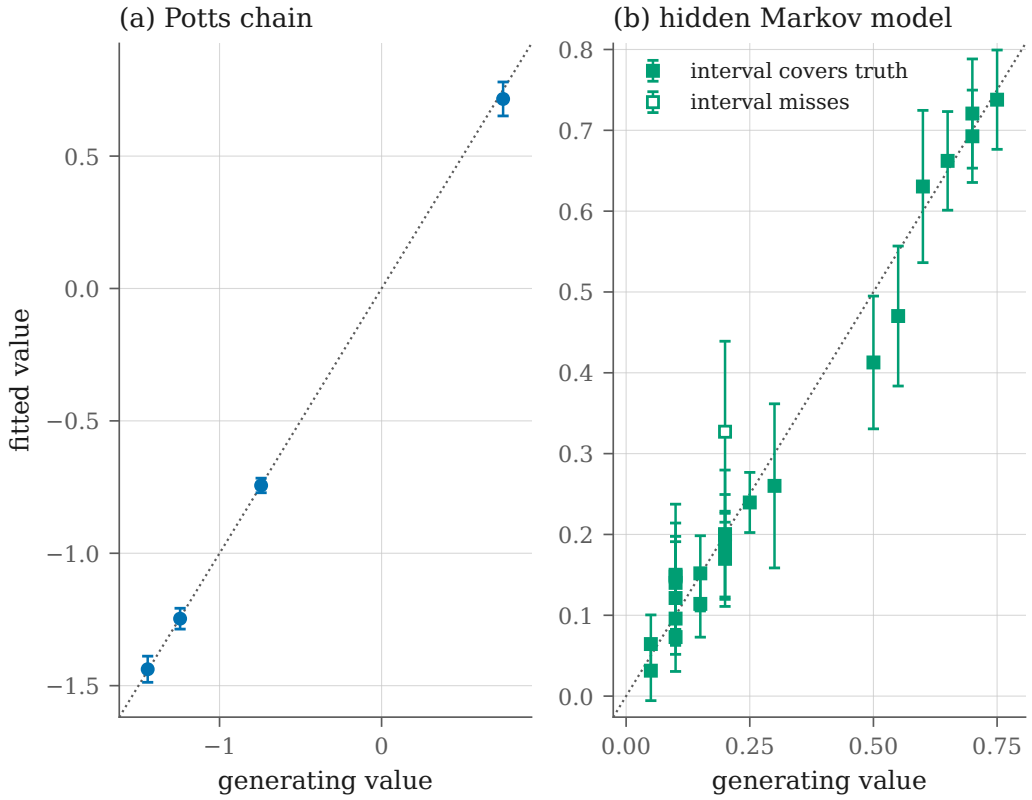

Figure 3: Parameter recovery for the two reference instances of the optimization interface, fitted by the same model-agnostic optimizer. Bars are 95 percent Wald intervals from the observed information, propagated to the reported parameters by the delta method; the dotted line is y = x. (a) A 3-state Potts chain, 400 chains of length 12, seed 20260902: coupling and gauge-fixed field, 4 of 4 intervals covering (1.00). (b) A 3-state, 4-symbol hidden Markov model, 600 sequences of length 15, seed 20260903: initial, transition and emission probabilities after aligning the hidden-state permutation the likelihood is invariant to, 23 of 24 intervals covering (0.96). Coverage on a single dataset is a draw, not a rate: the nominal 0.95 is checked over independent replicates by the fitting regression suite, and against sample size in the companion coverage figure.

Taxa	Sites	Simulate (MB)	Evaluate (MB)	Total (MB)
20	2000	0.624	2.43	3.06
20	11_000	3.43	13.4	16.8
100	11_000	17.5	69.7	87.2
1000	11_000	176	703	879

Table 1: Memory held while simulating a Jukes-Cantor alignment and while evaluating its likelihood by pruning, at 4 states, across the declared scale. The simulator retains every node's states and pruning retains one partial likelihood per open node, so both are computed from the arrays' own shapes and are exact rather than sampled; the regression suite pins each figure against the allocator's count. The last row is the declared maximum of 1000 taxa and 11_000 sites, which sits a factor of 20 inside the 16 GB unified-memory requirement. The topology is a caterpillar at every size, the deepest tree on its leaves and so the worst case: a balanced tree of the same taxon count costs strictly less. Timings are reported in the benchmark suite instead, because a wall clock ranks the machine rather than the code.

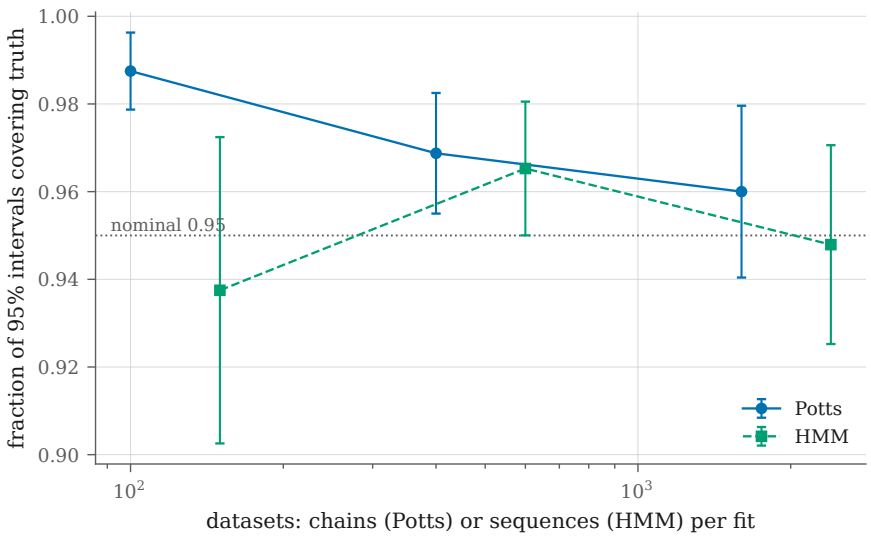

Figure 4: Realized coverage of the 95 percent Wald intervals built from the observed information, against the amount of data each fit sees. Bars are binomial standard errors on the realized fraction. Both instances converge on the nominal rate and neither sits on it at the smallest sample, which is what an asymptotic approximation does and a wrong formula does not. They approach from opposite sides. The Potts chain over-covers and comes down: $158/160 = 0.988$ at 100 chains, $96/100 = 0.960$ at 1600. The hidden Markov model under-covers and comes up: $45/48 = 0.938$ at 150 sequences, $91/96 = 0.948$ at 2400. The under-coverage has two identifiable sources absent from the Potts chain – some emission probabilities are fitted near zero, where a Wald interval on the log scale is a poor approximation, and aligning the hidden-state permutation to the truth is a post-selection step that costs a little coverage. Seeds are fixed, so every point is reproducible rather than redrawn. At the smallest hidden Markov size 6 of 18 fits reached the boundary of the parameter space – an emission probability estimated at zero – where the observed information is singular and there is no interval to report. Those fits are excluded from the counts above and stated here rather than dropped silently, since excluding them unannounced would select for the well-behaved samples.

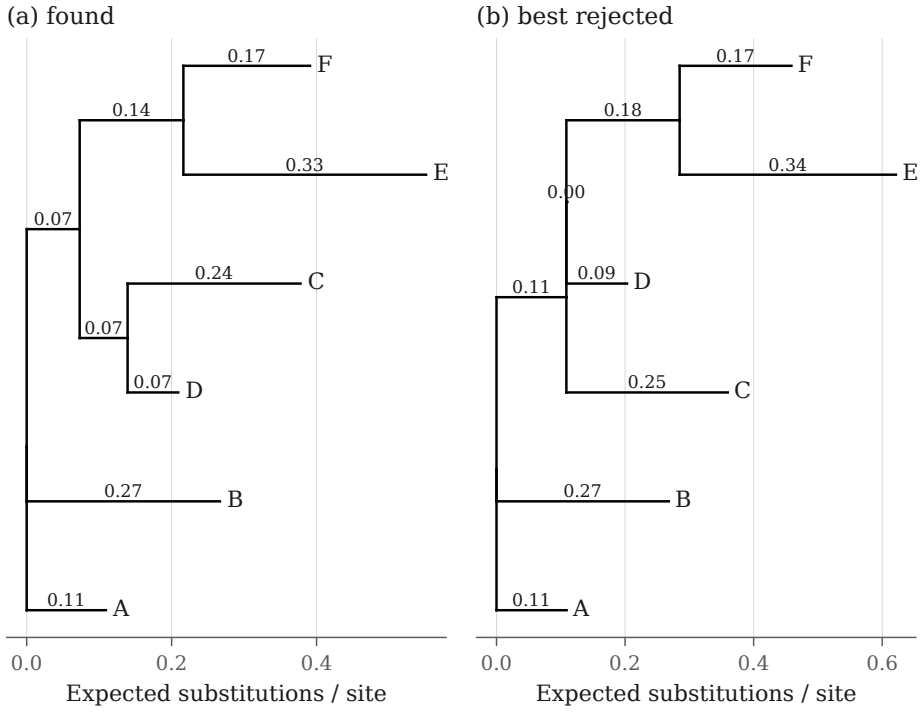

Figure 5: The topology hill climbing selected, beside the highest-scoring one it rejected, each with its own branch lengths fitted. Fixture: 4-state Jukes-Cantor over 1500 sites, seed 20260905. (a) scores -8775.5 and is the generating topology; (b) scores -8817.1, a difference of 41.6 log units. They differ by one split: C D are a group in one and not in the other. On this fixture the search recovers the generating tree, so a drawing of found against true would show the same tree twice; the runner-up is what the likelihood actually had to discriminate against. Note its shortest internal branch, fitted at 0.000: a wrong grouping is generally only supportable by collapsing the edge that would create it.

PLACEHOLDER: RL policy convergence against hill climbing

Figure 6: Learning curves of the reinforcement-learning proposal policy across $n \in \{10, 50, 100\}$, measured in improvement per objective-evaluation budget.

Figure 7: Wall-clock convergence against IQ-TREE 2 and RAxML-NG on empirical benchmark datasets.

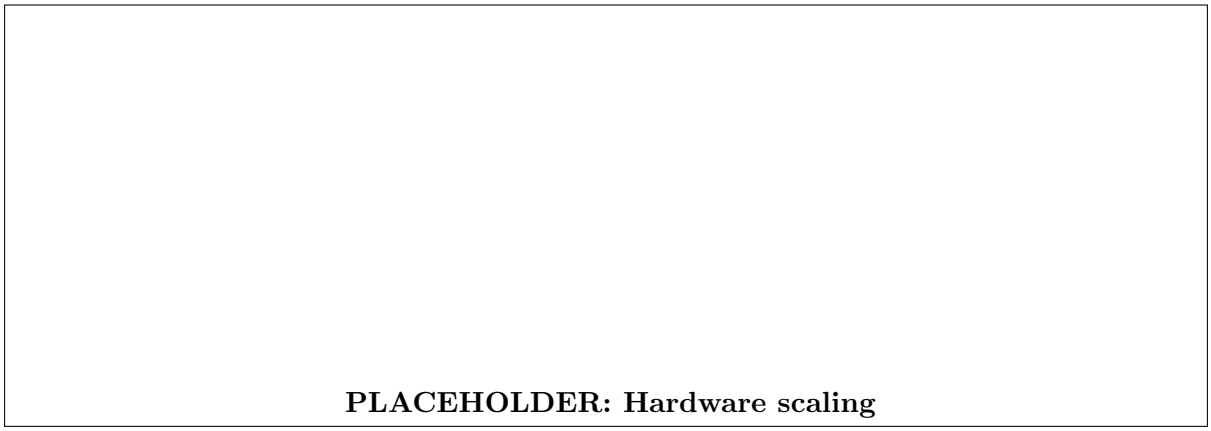

Figure 8: Throughput in nodes evaluated per second across CPU, Apple Silicon, and CUDA backends.

References

[1] Shun-ichi Amari. *Information Geometry and Its Applications.* Springer, Tokyo, 2016.

[2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning.* MIT Press, Cambridge, Massachusetts, 2016.

[3] David J. C. MacKay. *Information Theory, Inference, and Learning Algorithms.* Cambridge University Press, Cambridge, 2003.

[4] Marc Mézard and Andrea Montanari. *Information, Physics, and Computation.* Oxford University Press, Oxford, 2009.

[5] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization.* Springer, 2nd edition, 2006.

[6] Simon J. D. Prince. *Understanding Deep Learning.* MIT Press, Cambridge, Massachusetts, 2023.

[7] Kenneth H. Rosen. *Discrete Mathematics and Its Applications.* McGraw-Hill, 8th edition, 2019.

[8] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction.* MIT Press, Cambridge, Massachusetts, 2nd edition, 2018.